

Proposal: Multitenant Monorepo for TIN Products

Status: **Draft for approval** Date: 2026-10-05 Related: docs/superpowers/plans/2026-10-05-captable-multitenancy.md

1. Decision requested

Approve this direction:

- 1. Move Captable into a pnpm monorepo as the first **product app**.
- 2. Treat every future product (CRM, etc.) as another app in the same repo.
- 3. Treat every **client as a tenant** (a row in the database), not as a separate app or deployment.
- 4. Keep the Hubs as separate identity services. Products consume the tenant id from the Hub.
- 5. Extract shared packages only when a second product needs them.

2. Terms

Term	Meaning
Product app	A deployable application (Captable, a future CRM). One deployment serves all clients.
Client / tenant	A customer organisation. In Captable today this is a Company .
Hub	Separate identity services (shared-identity-hub , tin-boss-api , shared-client-care-hub).

3. Target structure

```
/
├── pnpm-workspace.yaml      # apps/* and packages/*
├── package.json            # root scripts, biome, husky
├── apps/
│   ├── captable/          # today's app, moved unchanged
│   └── <next-product>/    # added when needed
├── packages/
│   ├── tenancy/           # tenantDb + withTenant (from the current branch)
│   ├── auth/              # session + Hub token handling
│   ├── rbac/              # roles and policies
│   └── ui/, db/           # shared UI and database helpers
```

4. Why this shape

Clients are tenants, not apps. One app per client means N deployments, N databases and N migrations to run. That does not scale operationally. With shared products, a release ships once for every client, and a new client is a row, not a project.

Products are apps. Each product has its own data model, release cadence and deploy. A monorepo lets them share code and tooling without forcing them to ship together.

Isolation is enforced in code, once. Captable's multitenancy branch already moves tenant filtering out of individual procedures and into one scoped Prisma client (`tenantDb`) and one base procedure (`withTenant`). That layer is the natural first shared package.

Identity lives in the Hub. The Hub owns who a person is and which tenant a customer is. Each product owns what a member may do inside its own data. This is the split the multitenancy plan already uses:

Concern	Owner
Identity of a person, SSO, platform operators	Hub
Which tenant a customer is (<code>tenant_id</code>)	Hub (products store <code>hubTenantId</code>)
Membership, role, status inside a product	The product
What a role may do	The product
Tenant-scoped data access	Shared <code>tenancy</code> package

Extract late. Packages are created when a second product needs the code. Extracting earlier means guessing the shared interface and slows Captable down.

5. What this does not do

- It does not change Captable's behaviour. Step 2 below is a file move only.
- It does not merge the Hub repos into this repo. They are identity infrastructure with a different deploy and security profile, so they stay separate services.
- It does not make Captable depend on the Hub today. Hub linkage is gated (section 7).

6. Rollout

Step	Work	Risk
1	Merge <code>feat/multitenancy-phase0</code> into <code>main</code>	Low. Avoids rename conflicts with the in-flight router changes.
2	Restructure: move the app to <code>apps/captable</code> , add the workspace	Low if done as a pure move on a new branch. Needs CI, Docker, Fly and path configs updated.
3	Finish multitenancy Phases 2-3 (roles on mutations, tenant switching, bucket ownership)	Medium. Already planned.
4	Close the Hub gaps (section 7)	Depends on Hub team capacity.
5	Add the second product; extract <code>tenancy</code> and <code>auth</code> packages	Medium. First real test of the shared interface.

Steps 2 and 4 are independent and can run in parallel.

7. Hub readiness (open dependency)

Taken from the multitenancy plan, section 5. I have **not** inspected the Hub repos directly; this is the plan's audit, and it should be confirmed before relying on it.

- `shared-identity-hub` : unaudited `x-tenant-id` header bypass for tenantless platform admins; four competing definitions of "operator"; tenant switching and the Next.js adapter are specified but unbuilt.
- `tin-boss-api` : no tenant or membership read endpoint or webhook; no JWKS endpoint; no claims revocation story.
- Open decision: which identity provider the Hub fronts for login (Supabase Auth or WorkOS).

Until these exist, products keep their own sign-in (NextAuth) and store `hubTenantId` and `hubUserId` as unused columns, ready for linkage.

8. Risks

Risk	Mitigation
Monorepo restructure breaks deploys (Docker, Fly, CI paths)	Do it as a pure file move on its own branch; verify build, tests and the Docker image before merging.
Shared packages designed too early	Extract only when a second product needs the code.
Tenant isolation bug leaks data across clients	Structural scoping plus the cross-tenant test matrix and architecture guard already on the branch; Postgres RLS is planned as a deferred defense in depth.
Hub dependency delays products	Products stay standalone until the Hub gate opens.

9. Alternatives considered

Option	Why not
One app or deployment per client	Operational cost grows with every client; releases and migrations multiply.
Separate repo per product	Duplicates tenancy, auth and RBAC code, which drift apart.
Fold the Hub repos into the monorepo	Mixes identity infrastructure with product code; different security and deploy needs.
Do nothing (Captable stays single-package)	Fine for one product, but the second product would have to copy the tenancy code.

10. Open questions for approval

1. Confirm "each client is a tenant in shared product apps" is the intended model, not one deployment per client.
2. Which product comes second? It decides which shared packages get extracted first.
3. Identity provider for the Hub: Supabase Auth or WorkOS?
4. Who owns the Hub gap work in section 7?